Introduction to Message Passing Interface

F1_MA_HPC_1-24

Graded Exercise 1

Rules of the game

The exercise must be submitted by email to the Teacher (matteo.barborini@uni.lu) in a compressed directory named accordingly: `IntroToMPI_Exc1_{NAME}_{SURNAME}.zip`

The exercise must be submitted by 8:00 a.m. on the 21st of October 2026.

The directory must include:

1. the complete C code and a clean compilation file (and a README on how to compile) with all the details and dependencies;
2. output of the calculations done with the AION HPC of uni.lu;
3. a two page report (in pdf format) including 3 sections: 1. Introduction (Brief description of the exercise); 2. Explanation of the parallelization; 3. Explanation of the scaling results (Including plots of the Speed Up or of the Efficiency).

Basic Theory

We consider two matrices: a matrix $\mathbf{A}$ with N rows and M columns, and a matrix $\mathbf{B}$ with M rows and K columns both containing double precision elements.

We define the **matrix-matrix product** between $\mathbf{A}$ and the matrix $\mathbf{B}$ as the matrix $\mathbf{C} = \mathbf{AB}$ of N rows and K columns, of which the single element is defined as

$$\mathbf{C}_{kl} = \sum_{i=1}^{M} \mathbf{A}_{ki}\mathbf{B}_{il}, \tag{1}$$

where k is the index that goes through the rows of A (and C) and l is the index that goes through the columns of B (and C).

We define the **matrix-vector product** between the matrix $\mathbf{A}$ with N rows and M columns and the vector $\mathbf{b}$ of M elements, as the vector $\mathbf{c} = \mathbf{Ab}$ with N elements, whose single element is defined as

$$\mathbf{c}_i = \sum_{j=1}^{M} \mathbf{A}_{ij}\mathbf{b}_j, \tag{2}$$

where i is the index that runs over the N rows of the matrix $\mathbf{A}$ (and N elements of vector $\mathbf{c}$).

We define the **dot product** between two vectors $\mathbf{a}$ and $\mathbf{b}$ of N elements each as

$$d = \mathbf{a}^\top \cdot \mathbf{b} = \sum_{i=1}^{N} a_i b_i, \tag{3}$$

and the **norm** of a vector $\mathbf{a}$ with $\mathcal{N}$ elements as:

$$\mathcal{N} = \sqrt{\mathbf{a}^\top \cdot \mathbf{a}} = \sqrt{\sum_{i=1}^{N} a_i^2}. \tag{4}$$

Exercise

Following the example 5.1.MPI_dgemv.c that you can find in Lesson5 (Examples/Lesson5) of the repository (https://github.com/mbarborini/IntroductionToMPI) that requires the libraries

`array.h`, `linearalgebra.h`, and `mt19937ar.h` in Examples/Libraries of the same repository complete the following exercise.
Initialize:

1. a double precision matrix $\mathbf{A}$ with $N = 128000$ rows and $M = 500$ columns packed with random numbers between [-1:1] according to the function (`void init_random_matrix(int seed,int *n, int *m, double **A)`) in the `array.h` library. Use random `seed=10`. The library also requires `mt19937ar.h`.
2. a double precision matrix $\mathbf{B}$ with $M = 500$ rows and $K = 1000$ columns packed with random numbers between [-1:1] according to the function (`void init_random_matrix(int seed,int *n, int *m, double **A)`) in the `array.h` library. Use random `seed=20`. The library also requires `mt19937ar.h`.
3. a double precision vector $\mathbf{c}$ with $K = 1000$ elements with random numbers between [-1:1] according to the function (`void init_random_vector(int seed, int *n, double *a)`) in the `array.h` library. Use random `seed=30`. The library also requires `mt19937ar.h`.

Compute the operations:

$$\mathbf{d} = \mathbf{ABc}, \tag{5}$$

and subsequently compute the norm of the vector $\mathbf{d}$ (of N elements).
Using MPI parallelize the code using any type of strategy you prefer, and the functions we have learnt so far `MPI_*Send/MPI_*Recv`, `MPI_*Bcast`, `MPI_*Reduce`, and `MPI_*Allreduce`.
Measure the times for "Initialization", "Matrix operations", snd "Reduce Operations" with the MPI function `MPI_Wtime()`. (See example 5.1.MPI_dgemv.c in Lesson5)

At the end of the code print through rank 0:

1. the result for the norm of vector $\mathbf{d}$;
2. the timings for the "Initialization", "Matrix operations", "Reduce Operations", and the "Total Time" (See example 5.1.MPI_dgemv.c in Lesson5).

Run the calculation on the AION HPC for [1,2,4,8,16,32,64,128] MPI processes on the reservation (`mpi_course`) using as an example the script

```
#!/bin/bash -l
#SBATCH -N 1
#SBATCH --ntasks-per-node 128 # MPI processes per node
#SBATCH --cpus-per-task 1 # Cores per MPI process (OpenMP)
#SBATCH --time=0-01:00:00
#SBATCH -p batch
#SBATCH --reservation=mpi_course
#SBATCH --exclusive

module load toolchain/foss
export OMP_NUM_THREADS= $SLURM_CPUS_PER_TASK

for i in 1 2 4 8 16 32 64 128 ; do
        srun -n $i ${your_mpi_application} > out_$i
done
```

Submit the scripts using the command

```
sbatch -J name_of_my_job my_script.sh
```

After having collected all the results write a 2 pages report, including:

1. Introduction (Brief description of the exercise);
2. Explanation of the parallelization;
3. Explanation of the scaling results (Including a table of the timings and plots of the Speed Up or of the Efficiency).